

MERN Full Stack Developer: Assessment Task

Task Overview

Aspect	Details
Task Type	Full Stack Development
Time Allowed	4 Hours
Technology Stack	React.js (frontend), Node.js + Express.js (backend - optional), MongoDB (optional)
Submission Format	GitHub repository link + deployed application URL
Evaluation Criteria	Code quality, functionality, UI/UX, problem-solving

Task Description

You are required to build a "**Parental Legacy & Life Factors Calculator**" – a web application that takes a user's **Date of Birth** as input and automatically generates life factor values based on that date.

Core Requirement:

- **Mother Value + Father Value = Total for each factor**
- **Sum of all Mother values + Sum of all Father values = 100**

How It Works

Input

- User enters their **Date of Birth** (DD/MM/YYYY)

Processing

- Based on the **Day** of the month:
- **Odd Days (1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31):** Mother values are higher
- **Even Days (2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30):** Father values are higher
- **For each factor: Mother Value + Father Value = Total Value**
- **Grand Total: Sum of Mother values + Sum of Father values = 100**

Output

- Display all life factor values (Mother, Father, Total) in a structured format
- Show totals and visualizations
- Indicate which parent has a higher legacy value

Factor Ranges

Factor	Minimum	Maximum
Genetic Inheritance	9.333	10.777
Constitutional Vitality	8.111	9.111
Mental Patterns	6.111	7.111
Intellectual Capacity	6.333	6.999
Emotional Foundation	7.111	7.999
Spiritual Lineage	5.011	6.011
Soul Connections	5.111	6.222

Requirements

Must Have

Requirement	Description
DOB Input	Date picker for selecting Date of Birth
Validation	Ensure date is valid and not in the future
Auto-Calculation	Values are auto-generated on date selection
Value Display	Show Mother, Father, and Total for each factor
Totals	Show Mother Total, Father Total, Grand Total (= 100)
Parental Legacy	Display which parent has higher values
Charts	Visual comparison of Mother vs Father values
Responsive Design	Works on desktop and mobile

Nice to Have (Bonus)

Feature	Points
Export Results as PDF	+5
Export Results as CSV	+5
Dark/Light Mode Toggle	+5
Save Results (localStorage or MongoDB)	+10
User Authentication (JWT)	+10

Technical Constraints

Constraint	Requirement
Frontend	React.js (functional components, hooks)
State Management	useState / useReducer (Context API or Redux optional)
Styling	CSS3, SCSS, Tailwind, or Bootstrap
Charts	Chart.js, Recharts, or any library
Backend	Optional (can be in-memory or localStorage)
Database	Optional (localStorage or MongoDB)

Evaluation Criteria

Criteria	Weightage	Description
Code Quality	25%	Clean, readable, well-structured code
Functionality	25%	All features working as expected
Calculation Logic	20%	Correct generation of values with total = 100
UI/UX Design	15%	Professional, responsive, and user-friendly
Charts	10%	Clear visualization of data
Documentation	5%	Clear README with setup instructions

Submission Guidelines

Step	Action
------	--------

- | | |
|---|--|
| 1 | Create a GitHub repository with your project |
| 2 | Push your code with proper commit messages |
| 3 | Create a README.md file with project setup instructions |
| 4 | Deploy your application (optional but recommended) |
| 5 | Share the repository link and deployed URL via email to WhatsApp: 8454815742 |
| 6 | Ensure your code is complete within the 4-hour timeframe |

Frequently Asked Questions

Question	Answer
Do I need a backend?	No, you can use React with localStorage
Do I need a database?	No, localStorage is sufficient for this task
Can I use additional libraries?	Yes, as long as they enhance the functionality
Can I skip charts?	No, charts are required for visualization
What if I can't finish in 4 hours?	Submit what you have completed; quality over quantity
How will my task be evaluated?	Based on code quality, functionality, and UI/UX